Cálculo Relacional

Introducción

Hemos destacado las características del álgebra relacional que precisa que se especifique la secuencia ordenada de operadores a aplicar para obtener un resultado. El orden influye y puede ser tenido en cuenta como estrategia para mejorar la performance de la búsqueda.

Ahora analizaremos el cálculo relacional, lenguaje no procedural que sólo precisa especificar qué se desea obtener y no cómo. El sistema realizará las juntas, proyecciones, etc. El cálculo relacional es un lenguaje declarativo.

Cabe destacar que las potencias expresivas del *álgebra relacional* y del *cálculo relacional seguro* (*safe relational calculus*) son equivalentes ya que cualquier consulta expresada en uno de ellos puede realizarse en el otro.

1. Calculo Relacional

Es un lenguaje de consulta declarativo y no procedural. Se basa en el cálculo de predicados de primer orden de la lógica matemática. Utiliza variables, constantes, operadores de comparación, conectivos lógicos, cuantificadores.

Los hay de dos tipos, según cuales sean sus variables: cálculo relacional de tuplas y cálculo relacional de dominios.

1.1 Cálculo Relacional de Tuplas (Tuple Relational Calculus, TRC)

Utiliza variables, donde cada variable sirve para representar cada una de las tuplas de cierta relación. Las variables usadas sólo representan tuplas.

Una consulta corresponde al recupero de ciertas tuplas, por lo tanto se expresa como en el cálculo de predicados de primer orden, a través del conjunto de tuplas que satisfacen cierta fórmula:

{ T | fórmula(T) } (expresión 1)

Aquellas tuplas que satisfacen la fórmula son las que se recuperan en la consulta. Una fórmula se construye a partir de átomos. Comencemos definiendo los átomos para el cálculo relacional de tuplas.

Definición de Átomo

- Si **r** es una relación y **T** es una variable, entonces **r(T)** es un átomo. Este átomo se usa para referir a cada una de las tuplas de dicha relación **r**. O sea, r(T) se interpreta como $T \in r$.
- Sean **U** y **V** variables, **i** un atributo de la relación que representa **U** , **j** un atributo de la relación que representa **V** (ambos atributos con dominios compatibles), entonces **U[i] op V[j]** es un átomo, donde el **op** puede ser $\{<, \leq, >, \geq, = \text{ y } \neq\}$.
- Sean **U** una variable, **i** un atributo de la relación que representa **U**, **c** una constante en el dominio del atributo **i**, y **op** alguno de los operadores $\{<, \leq, >, \geq, = \text{ y } \neq\}$, entonces **U[i] op c** es un átomo.

Silvia Alicia Gómez & Leticia Irene Gómez

Las variables que figuran en las fórmulas pueden ser libres (globales) o ligadas (locales). Las ligadas son las que están dentro del alcance de un cuantificador existencial o universal: ∃ (existe) o ∀ (para todo).

Definición de Fórmula

- Un átomo es una fórmula. Las variables de la fórmula son libres.
- Si **f1** y **f2** son fórmulas entonces también son fórmulas:

  f1 ∨ f2 (or)
  f1 ∧ f2 (and)
  ¬f1 (not)

En estos casos las variables siguen libres o ligadas según como lo eran en cada una de las fórmulas **f1** y **f2.**

- Si **f** es fórmula, entonces **(∃U) (f_u)** es fórmula. La variable **U** está ligada en esa fórmula
- Si **f** es fórmula, entonces **(∀U) (f_u)** es fórmula. La variable **U** está ligada en esa fórmula

Importante

Las únicas variables libres que pueden aparecer en una expresión del cálculo relacional de tuplas son aquellas que aparecen en la izquierda del pipe (ver la expresión 1 en la página 1).

Podemos decir que "construimos" las tuplas de la relación resultante de la consulta.

1.1.1 Fórmulas Seguras

Hay que tener cuidado de no expresar fórmulas que tengan un numero infinito de tuplas. Eso ocurre cuando se realiza una consulta al estilo **{ T | ¬ r (T) }**donde r es una relación.

Así como en programación se descartan los ciclos infinitos, porque no son algoritmos, en base de datos no interesan las consultas de este tipo ya que ***queremos devolverle al usuario una consulta en un intervalo finito de tiempo***. La anterior es una fórmula no segura porque las tuplas que no figuran en la relación r son infinitas (ni siquiera están representadas en la base de datos), o sea la ***relación resultante no es finita***. O sea, aunque sintácticamente correcta, consultas que devuelvan este tipo de relaciones no son interesantes.

Para evitar este problema vamos a aclarar a qué se denomina dominio de una fórmula y cuándo una fórmula se considera segura.

Dominio Activo de una BD (active domain)

Son los valores que ocurren en las relaciones de la BD.

Silvia Alicia Gómez & Leticia Irene Gómez

Dominio de una Fórmula

Son los valores constantes que aparecen en la fórmula y los valores que los atributos toman en las tuplas de todas las relaciones que aparecen mencionadas en dicha fórmula. O sea, valores constantes y el subconjunto del dominio activo que aparece en la fórmula.

Ejemplo:

Dada la fórmula **r(T) ∧ T[x] =5** el dominio de dicha fórmula es el conjunto formado por el número **5**, y todos los valores que aparecen en las distintas tuplas de la relación **r**.

Fórmula Segura

Una expresión **{ T | f(T) }** es segura si para toda BD, todos los valores que aparecen en el resultado son valores del dominio de **f**.

Fórmula Segura (otra definición)

Una expresión **{ T | f(T) }** es segura si para toda BD sólo puede formular consultas que no extiendan el dominio activo de la BD.

Ejemplo:

La consulta **{ T | ¬ r (T) }** no es segura porque el dominio activo de **r (T)** es el conjunto de los valores que aparecen en las distintas tuplas de la relación **r**, pero los valores que se obtendrían al ejecutarla son precisamente su complemento, o sea se obtienen valores que no son parte del dominio activo.

Ejemplo:

La consulta **{ T | alumno(T) ∧ (∃ M) (alumno_1999(M) ∧ ¬ alumno_2000(M) ∧ M[legajo]=T[legajo]) }** es segura. Ante la presencia de una negación podría pensarse que es insegura (se habla de elementos que no están en una relación). Pero gracias al binding de M con respecto a alumno_1999 estamos frente a una fórmula segura. Lo que se obtiene al ejecutarla son las tuplas que están en las relaciones **alumno** y **alumno_1999** pero no están en la relación **alumno_2000**. O sea, lo obtenido es un subconjunto del dominio activo.

Importante

Prestar atención y replantear las fórmulas que usan **¬relación(Variable)**. Usar leyes de la lógica para transformar una expresión de este tipo en otra y verificar que las fórmulas que se expresan sean seguras.

Puede servir recordar: "transformar un tipo de cualificador por la negación del otro, y acompañarlo por la negación de la fórmula "

- $(\forall X)\ (p_X)$ se puede reemplazar por $(\neg\exists X)\ (\neg p_X)$
- $(\exists X)\ (p_X)$ se puede reemplazar por $(\neg\forall X)\ (\neg p_X)$

Silvia Alicia Gómez & Leticia Irene Gómez

1.1.2 Equivalencia entre Algebra relacional y Cálculo Relacional de tuplas

Vamos a mostrar la equivalencia reescribiendo los operadores:

Algebra Relacional	*Calculo Relacional de Tuplas*
r ∪ s	**{ T | r(T) ∨ s(T) }** donde **r** y **s** tienen el mismo grado (y dominio compatibles)
r × s	**{ T | (∃ U) (r(U) ∧ (∃ V) (s(V) ∧ T[1]=U[1] ∧ ... T[n]=U[n] ∧ T[n+1]=V[1] ∧... T[n+m]=V[m])) }** donde **r** tiene grado **n**, **s** tiene grado **m** y **t** tiene grado **n+m**,
r – s	**{ T | r(T) ∧ ¬s(T) }** donde **r** y **s** tienen el mismo grado (dominio compatibles) ¿Por qué es segura?
$\pi_{A1, A2..Ak}$(r)	**{ T | (∃ U) (r(U) ∧ T[1]=U[1] ∧ ... T[k]=U[k]) }** donde **r** tiene grado **k** y la relación resultante tiene grado, con **n >= k**
σ_{cond} (r)	**{ T | r(T) ∧ cond' }** ***cond'*** es equivalente a ***cond*** pero con distinta sintaxis.

1.1.3 Ejemplos de Cálculo Relacional de Tuplas

A) Supongamos la siguiente relación **alumno**:

alumno	
legajo	**nombre**
400	**'Ana Lopez'**
1100	**'Juan Perez'**
750	**'Sofía Sosa'**
1300	**'Raul Sosa'**
2000	**'Ana Lopez'**

Si se quisiera obtener los alumnos cuyos legajos sean mayores que 1000 y menores o iguales a 1900 entonces se podía indicar:

{ T | alumno(T) ∧ T[legajo] > 1000 ∧ T[legajo]≤ 1900 }

nueva relacion	
legajo	**nombre**
1100	**'Juan Perez'**
1300	**'Raul Sosa'**

Silvia Alicia Gómez & Leticia Irene Gómez

B) Sea la siguiente relación **alumno**:

alumno	
legajo	nombre
400	'Ana Lopez'
1100	'Juan Perez'
750	'Sofía Sosa'
1300	'Raul Sosa'
2000	'Ana Lopez'

Si se quisiera obtener sólo los nombres de los alumnos, tendríamos que indicar:

{ T | (∃ U) (alumno(U) ∧ T[nombre]=U[nombre]) }

nueva relacion
nombre
'Ana Lopez'
'Juan Perez'
'Sofía Sosa'
'Raul Sosa'

C) Si se tienen las relaciones **alumno** y **examen**:

alumno	
legajo	nombre
400	'Ana Lopez'
1100	'Juan Perez'
750	'Sofía Sosa'
1300	'Raul Sosa'
2000	'Ana Lopez'

examen		
legajo	nroActa	nota
750	13.33	9
400	11.03	3
400	11.04	7
400	11.08	8
750	11.03	8
2000	11.03	2

para hacer **alumno** x **examen**, en el calculo relacional de tuplas haríamos:

{ T | (∃ U) (alumno(U) ∧ (∃ V) (examen(V) ∧ T[legajo]=U[legajo] ∧ T[nombre]=U[nombre] ∧ T[legajo2]=V[legajo] ∧ T[nroActa]=V[nroActa] ∧ T[nota]=V[nota])) }

nueva relacion				
legajo	nombre	examen.legajo	nroActa	nota
400	'Ana Lopez'	750	13.33	9
400	'Ana Lopez'	400	11.03	3
400	'Ana Lopez'	400	11.04	7
400	'Ana Lopez'	400	11.08	8
400	'Ana Lopez'	750	11.03	8
400	'Ana Lopez'	2000	11.03	2
1100	'Juan Perez'	750	13.33	9
1100	'Juan Perez'	400	11.03	3
1100	'Juan Perez'	400	11.04	7
1100	'Juan Perez'	400	11.08	8
1100	'Juan Perez'	750	11.03	8
1100	'Juan Perez'	2000	11.03	2

Silvia Alicia Gómez & Leticia Irene Gómez

750	'Sofía Sosa'	750	13.33	9
750	'Sofía Sosa'	400	11.03	3
750	'Sofía Sosa'	400	11.04	7
750	'Sofía Sosa'	400	11.08	8
750	'Sofía Sosa'	750	11.03	8
750	'Sofía Sosa'	2000	11.03	2
1300	'Raul Sosa'	750	13.33	9
1300	'Raul Sosa'	400	11.03	3
1300	'Raul Sosa'	400	11.04	7
1300	'Raul Sosa'	400	11.08	8
1300	'Raul Sosa'	750	11.03	8
1300	'Raul Sosa'	2000	11.03	2
2000	'Ana Lopez'	750	13.33	9
2000	'Ana Lopez'	400	11.03	3
2000	'Ana Lopez'	400	11.04	7
2000	'Ana Lopez'	400	11.08	8
2000	'Ana Lopez'	750	11.03	8
2000	'Ana Lopez'	2000	11.03	2

D) Sean las siguientes relaciones **alumno** y **examen**

alumno	
legajo	nombre
400	'Ana Lopez'
1100	'Juan Perez'
750	'Sofía Sosa'
1300	'Raul Sosa'
2000	'Ana Lopez'

examen		
legajo	nroActa	nota
750	13.33	9
400	11.03	3
400	11.04	7
400	11.08	8
750	11.03	8
2000	11.03	2

Si se quisiera obtener lo mismo que cuando en el algebra relacional hacíamos **alumno ▷◁ examen**, en el cálculo relacional de tuplas haríamos:

{ T | (∃ U) (alumno(U) ∧ (∃ V) (examen(V) ∧ U[legajo]=V[legajo] ∧ T[legajo]=U[legajo] ∧ T[nombre]=U[nombre] ∧ T[nroActa]=V[nroActa] ∧ T[nota]=V[nota])) }

nueva relacion			
legajo	nombre	nroActa	nota
400	'Ana Lopez'	11.03	3
400	'Ana Lopez'	11.04	7
400	'Ana Lopez'	11.08	8
750	'Sofia Sosa'	13.33	9
750	'Sofia Sosa'	11.03	8
2000	'Ana Lopez'	11.03	2

Silvia Alicia Gómez & Leticia Irene Gómez

E) Sean las siguientes relaciones:

r	
datoA	datoB
10	'aa'
30	'bb'
20	'cc'
30	'cc'
30	'dd'
40	'cc'
20	'bb'

s
datoB
'bb'
'cc'

Para obtener lo mismo que el **r % s** del álgebra relacional, en el cálculo relacional de tuplas haríamos:

{ T | (∃ U) (r(U) ∧ (∀ V) (s(V) ⇒ (∃ W) (r(W) ∧ V[datoB]=W[datoB] ∧ U[datoA]=W[datoA])) ∧ U[datoA]=T[datoA]) }

Pero tenemos que transformar la expresión **c ⇒ d** en su equivalente **¬ c ∨ d.** Obteniendo:

{ T | (∃ U) (r(U) ∧ (∀ V) (¬ s(V) ∨ (∃ W) (r(W) ∧ V[datoB]=W[datoB] ∧ U[datoA]=W[datoA])) ∧ U[datoA]=T[datoA]) }

Es segura esta expresión del cálculo relacional de tuplas? Hay una variable que dice "los elementos que no son de la relación S". Reescribimos sabiendo que **(∀x) (Px)** es equivalente a **(¬∃x) (¬Px)**

{ T | (∃ U)(r(U) ∧ (¬∃ V) (s(V) ∧ (¬∃ W) (r(W) ∧ V[datoB]=W[datoB] ∧ U[datoA]=W[datoA])) ∧ U[datoA]=T[datoA]) }

Lo cual es una expresión segura.

r % s
datoA
30
20

F) Se quiere obtener los legajos de los alumnos que intentaron rendir todas las materias (aunque no las hayan aprobado), contando con las siguientes relaciones:

examen2			
legajo	NroActa	nota	nombreMateria
750	13.33	9	'Pgm I'
400	11.03	3	'BD'
400	11.04	7	'Pgm I'
400	11.08	8	'BD'
750	11.03	8	'BD'
2000	11.03	2	'BD'

materia
nombreMateria
'Pgm I'
'BD'

Silvia Alicia Gómez & Leticia Irene Gómez

La consulta se calcula:

{ T | (∃ U) (examen2(U) ∧ (¬∃ V) (materia(V) ∧ (¬∃ W) (examen2(W) ∧ V[nombreMateria]=W[nombreMateria] ∧ U[legajo]= W[legajo])) ∧ U[legajo]=T[legajo]) }

Notar que mientras en el álgebra relacional hay que hacer una proyección previa y calcular $\pi_{legajo,\ materia}$ **(examen2) % materia**, o sea hubo que indicar la secuencia explícita de primero calcular la proyección y recién ahí calcular la división, acá sólo hubo que indicar lo que queríamos obtener como tuplas resultantes.

nueva relacion
legajo
750
400

1.1.4 Ejercicios

Se tiene el siguiente esquema de base de datos:

proveedor		
nroProv	nombreProv	ciudad

producto	
codProd	descripcion

cliente		
codCli	nombreCli	ciudad

pedido			
codCli	codProd	nroProv	cantidad

Expresar con cálculo relacional de tuplas las siguientes consultas:

a) Listar los nombres de los proveedores que no viven en 'Santa Cruz'
b) Listar los nombres de los clientes que pidieron el producto con código 50
c) Listar los pares de nombres de clientes que viven en la misma ciudad
d) Listar los nombres de los proveedores y clientes que viven en 'Santa Cruz'

Respuestas

a) { T | (∃ U) (proveedor(U) ∧ U[ciudad] ≠'Santa Cruz' ∧ T[nombre]=U[nombreProv]) }

b) { T | (∃ V) (cliente(V) ∧ (∃ W) (pedido(W) ∧ V[codCli]=W[codCli] ∧ W[codProd]=50) ∧ T[nombre] = V[nombreCli]) }

c) { T | (∃ V) (cliente(V) ∧ (∃ W) (cliente(W) ∧ V[codCli] < W[codCli] ∧ V[ciudad]=W[ciudad] ∧ T[nombre1]=W[nombreCli]) ∧ T[nombre2]=V[nombreCli]) }

d) { T | (∃ U) (cliente(U) ∧ U[ciudad]='Santa Cruz' ∧ T[nombre]=U[nombreCli]) ∨ (∃ U) (proveedor(U) ∧ U[ciudad]='Santa Cruz' ∧ T[nombre]=U[nombreProv]) }

Silvia Alicia Gómez & Leticia Irene Gómez

1.2. Cálculo Relacional de Dominios (Domain Relational Calculus, DRC)

Utiliza variables, donde cada variable sirve para representar cada uno de los valores que pueden tomar los atributos de las tuplas de cierta relación. Las variables usadas sólo representan los atributos de las tuplas.

Una consulta corresponde al recupero de ciertas tuplas. La semántica de una consulta se expresa, como en el cálculo de predicados de primer orden, a través del conjunto de tuplas que satisfacen cierta fórmula, pero dichas tuplas ahora se expresa indicando la lista de variables que representan a cada atributo correspondiente a la nueva relación:

{ X1, X2, . . . , Xn | fórmula(X1,X2,....Xn) } (expresión 2)

Aquellas variables que satisfacen la fórmula son las que se recuperan en la consulta formando así las tuplas resultantes.

Una fórmula se construye a partir de átomos, pero los átomos se definen de forma diferente que en el cálculo relacional de tuplas.

Definición de Átomo (o Fórmula Atómica)

- Si **r** es una relación de grado n que pertenece al esquema **R** y cada uno de los **Xi** es una variable en el dominio del atributo i-ésimo del esquema R con $1 \leq i \leq n$, entonces **r(X1,X2,...,Xn)** es un átomo.
- Sean X**i** y **Xj** variables, entonces **Xi op Xj** es un átomo, donde el **op** puede ser $\{<, \leq, >, \geq, = \text{ y } \neq\}$. Obviamente los dominios correspondientes tendrían que tener compatibilidad para que la comparación tuviera sentido.
- Sean X**i** una variable, **c** una constante en el dominio de la variable X**i**, y **op** alguno de los operadores$\{<, \leq, >, \geq, = \text{ y } \neq\}$, entonces **Xi op c** es un átomo.

Las variables que figuran en las fórmulas pueden ser libres (globales) o ligadas (locales). Las ligadas son las que están dentro del alcance de un cuantificador existencial o universal: $\exists$ (existe) o $\forall$ (para todo).

Definición de Fórmula

- Un átomo es una fórmula. Las variables de la fórmula son libres.
- Si **f1** y **f2** son fórmulas entonces también son fórmulas:

  f1 $\vee$ f2 (or)
  f1 $\wedge$ f2 (and)
  $\neg$f1 (not)

En estos casos las variables siguen libres o ligadas según como lo eran en cada una de las fórmulas **f1** y **f2.**

- Si **f** es fórmula, entonces **($\exists$U) (f_U)** es fórmula. La variable u está ligada es esa fórmula.
- Si **f** es fórmula, entonces **($\forall$U) (f_U)** es fórmula. La variable u está ligada es esa fórmula.

Silvia Alicia Gómez & Leticia Irene Gómez

Importante

Las únicas variables libres que pueden aparecer en una expresión del cálculo relacional de dominios son aquellas que aparecen en la izquierda del pipe (ver expresión 2).

Podemos decir que "construimos" la respuesta a la consulta a través de sus atributos.

1.2.1 Fórmulas Seguras

Análogamente al cálculo relacional de tuplas, también hay que tener cuidado de no expresar fórmulas que tengan un número infinito de tuplas. En esto ocurriría en la siguiente consulta:

{ X1, X2, X3 | ¬ r(X1, X2, X3) } donde **r** es una relación

La anterior es una fórmula no segura porque las variables **X1, X2, X3** que permiten formar las tuplas que hay que obtener en el resultado de la consulta se obtiene analizando los valores que no figuran en la relación **r** y otra vez estamos ante un problema no resoluble en un tiempo finito. (estamos pidiendo armar todas las combinaciones de **X1 X2 X3** tal que alguno de ellos no estén en las tuplas de **r**).

Para evitar este problema vamos a definir qué se entiende por fórmula segura en el cálculo relacional de dominios

Definición

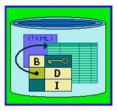
Una expresión **{ X1, X2, ..., Xn | F(X_1,X_2,..., X_n) }** es segura si se cumple las siguientes tres condiciones:

- Todos los valores que aparecen en las tuplas de la expresión son valores del dominio de **f** (dominio fue definido en el otro cálculo relacional)
- Para cada subfórmula tipo **(∃X) (f(X))** la subfórmula es verdadera si y sólo si hay un valor de **X** en el dominio de **f** tal que **f(X)** es verdadero
- Para cada subfórmula tipo **(∀X) (f(X))** la subfórmula es verdadera si y sólo **f(X)** es verdadero para todos los valores del dominio de **f**.

Aclaración

Como a veces hay que especificar un cuantificador para cada una de las variables, para ***simplificar la notación***, en vez de colocar **(∃X1) (∃X2) ... (∃Xn)** expresaremos **(∃ X1, X2, ..., Xn)**

Silvia Alicia Gómez & Leticia Irene Gómez



1.2.2 Ejemplos de Cálculo Relacional de Dominios

A) Supongamos la siguiente relación **alumno**:

alumno	
legajo	nombre
400	'Ana Lopez'
1100	'Juan Perez'
750	'Sofía Sosa'
1300	'Raul Sosa'
2000	'Ana Lopez'

Si se quisiera obtener los alumnos cuyos legajos sean mayores que 1000 y menores o iguales a 1900 entonces se podía indicar:

{ Legajo, Nombre | alumno(Legajo,Nombre) ∧ Legajo > 1000 ∧ Legajo≤ 1900 }

y se obtendría:

nueva relacion	
legajo	nombre
1100	'Juan Perez'
1300	'Raul Sosa'

B) Sea la siguiente relación **alumno**:

alumno	
legajo	nombre
400	'Ana Lopez'
1100	'Juan Perez'
750	'Sofía Sosa'
1300	'Raul Sosa'
2000	'Ana Lopez'

Si se quisiera obtener sólo los nombres de los alumnos:

{ Nombre | (∃ Legajo) (alumno(Legajo, Nombre)) }

nueva relacion
Nombre
'Ana Lopez'
'Juan Perez'
'Sofía Sosa'
'Raul Sosa'

C) Si se tienen las relaciones **alumno** y **examen**:

alumno	
legajo	nombre
400	'Ana Lopez'
1100	'Juan Perez'
750	'Sofía Sosa'
1300	'Raul Sosa'
2000	'Ana Lopez'

Silvia Alicia Gómez & Leticia Irene Gómez

examen		
Legajo	nroActa	nota
750	13.33	9
400	11.03	3
400	11.04	7
400	11.08	8
750	11.03	8
2000	11.03	2

Para hacer **alumno** x **examen**, en el cálculo relacional de dominios haríamos:

{ Legajo1, Nombre, Legajo2, NroActa, Nota |
alumno(Legajo1, Nombre) ∧ examen(Legajo2, NroActa, Nota) }

nueva relacion				
Legajo1	Nombre	Legajo2	NroActa	Nota
400	'Ana Lopez'	750	13.33	9
400	'Ana Lopez'	400	11.03	3
400	'Ana Lopez'	400	11.04	7
400	'Ana Lopez'	400	11.08	8
400	'Ana Lopez'	750	11.03	8
400	'Ana Lopez'	2000	11.03	2
1100	'Juan Perez'	750	13.33	9
1100	'Juan Perez'	400	11.03	3
1100	'Juan Perez'	400	11.04	7
1100	'Juan Perez'	400	11.08	8
1100	'Juan Perez'	750	11.03	8
1100	'Juan Perez'	2000	11.03	2
750	'Sofía Sosa'	750	13.33	9
750	'Sofía Sosa'	400	11.03	3
750	'Sofía Sosa'	400	11.04	7
750	'Sofía Sosa'	400	11.08	8
750	'Sofía Sosa'	750	11.03	8
750	'Sofía Sosa'	2000	11.03	2
1300	'Raul Sosa'	750	13.33	9
1300	'Raul Sosa'	400	11.03	3
1300	'Raul Sosa'	400	11.04	7
1300	'Raul Sosa'	400	11.08	8
1300	'Raul Sosa'	750	11.03	8
1300	'Raul Sosa'	2000	11.03	2
2000	'Ana Lopez'	750	13.33	9
2000	'Ana Lopez'	400	11.03	3
2000	'Ana Lopez'	400	11.04	7
2000	'Ana Lopez'	400	11.08	8
2000	'Ana Lopez'	750	11.03	8
2000	'Ana Lopez'	2000	11.03	2

D) Sean las siguientes relaciones **alumno** y **examen**:

alumno	
legajo	nombre
400	'Ana Lopez'
1100	'Juan Perez'
750	'Sofía Sosa'
1300	'Raul Sosa'
2000	'Ana Lopez'

Silvia Alicia Gómez & Leticia Irene Gómez

examen		
legajo	nroActa	nota
750	13.33	9
400	11.03	3
400	11.04	7
400	11.08	8
750	11.03	8
2000	11.03	2

Si se quisiera obtener lo mismo que cuando en el álgebra relacional hacíamos **alumno** $\triangleright\triangleleft$ **examen**, en el cálculo relacional de dominios haríamos:

{ Legajo, Nombre, NroActa, Nota |
alumno(Legajo, Nombre) $\wedge$ examen(Legajo, NroActa, Nota) }

nueva relacion			
Legajo	Nombre	NroActa	Nota
400	'Ana Lopez'	11.03	3
400	'Ana Lopez'	11.04	7
400	'Ana Lopez'	11.08	8
750	'Sofia Sosa'	13.33	9
750	'Sofia Sosa'	11.03	8
2000	'Ana Lopez'	11.03	2

E) Sean las siguientes relaciones

R	
datoA	datoB
10	'aa'
30	'bb'
20	'cc'
30	'cc'
30	'dd'
40	'cc'
20	'bb'

s
datoB
'bb'
'cc'

Para obtener lo mismo que el **r % s** del álgebra relacional, en el cálculo relacional de dominios haríamos:

{ DatoA | ($\exists$ DatoB) (r(DatoA, DatoB) $\wedge$
($\forall$ DatoD) (s(DatoD) $\Rightarrow$ r(DatoA, DatoD))) }

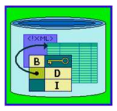
(el alcance de datoA debe llegar hasta al final porque lo usa en la última expresión)

Pero como en cálculo relacional no tenemos la implicación lógica ($\Rightarrow$), tenemos que reescribir la expresión: **p $\Rightarrow$ q** es equivalente a **$\neg$ p $\vee$ q.** Obteniendo,

{ DatoA | ($\exists$ DatoB) (r(DatoA, DatoB) $\wedge$
($\forall$ DatoD) ($\neg$s(DatoD) $\vee$ r(DatoA, DatoD)))}

Tenemos una negación de una relación. Como sabemos que **($\forall$X) (p(X))** es equivalente a **($\neg\exists$x) ($\neg$p(x))** entonces obtenemos:

Silvia Alicia Gómez & Leticia Irene Gómez



{ DatoA | (∃ DatoB) (r(DatoA, DatoB) ∧
(¬∃ DatoD) (s(DatoD) ∧ ¬r(DatoA, DatoD))) }

Aunque aparece una negación de r, sin embargo la variable datoA tiene binding en r y datoD en s (con un ∧**).** Es una expresión segura. De esta forma obtendríamos por respuesta:

r % s
DatoA
30
20

F) Se quiere obtener los legajos de los alumnos que intentaron rendir todas las materias (aunque no las hayan aprobado), contando con las siguientes relaciones:

examen2			
legajo	nroActa	nota	nombreMateria
750	13.33	9	'Pgm I'
400	11.03	3	'BD'
400	11.04	7	'Pgm I'
400	11.08	8	'BD'
750	11.03	8	'BD'
2000	11.03	2	'BD'

materia
nombreMateria
'Pgm I'
'BD'

La consulta se calcula:

{ Legajo | (∃ NomMat, NroActa, Nota) (examen2(Legajo, NroActa, Nota, NomMat) ∧
(∀ C) (materia(C) ⇒ (∃ M, N) (examen2(Legajo, M, N, C)))) }

O sea,
{ Legajo | (∃ NomMat, NroActa, Nota) (examen2(Legajo, NroActa, Nota, NomMat) ∧
(∀ C) (¬materia(C) ∨ (∃ M, N) (examen2(Legajo, M, N, C)))) }

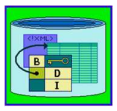
O sea,
{ Legajo | (∃ NomMat, NroActa, Nota) (examen2(Legajo, NroActa, Nota, NomMat) ∧
(¬∃ C) (materia(C) ∧ (¬ ∃ M, N) (examen2(Legajo, M, N, C)))) }

Notar que mientras en el álgebra relacional hay que hacer una proyección previa y calcular $\pi_{legajo,\ materia}$ (examen2) % materia, o sea hubo que indicar la secuencia explícita de primero calcular la proyección y recién ahí calcular la división, acá sólo hubo que indicar lo que queríamos.

Obviamente esto resalta el hecho de que el álgebra relacional es procedural, y en cambio el cálculo relacional es declarativo.

nueva relacion
Legajo
750
400

Silvia Alicia Gómez & Leticia Irene Gómez



1.2.3 Ejercicios

Se tiene el siguiente esquema de base de datos:

proveedor		
nroProv	nombreProv	ciudad

producto	
codProd	descripcion

cliente		
codCli	nombreCli	ciudad

pedido			
codCli	**codProd**	**nroProv**	cantidad

Expresar con cálculo relacional de tuplas las siguientes consultas

a) Listar los nombres de los proveedores que no viven en 'Santa Cruz'
b) Listar los nombres de los clientes que pidieron el producto con código 50
c) Listar los pares de nombres de clientes que viven en la misma ciudad
d) Listar los nombres de los proveedores y clientes que viven en 'Santa Cruz'

Respuestas:

a) { NombreProv |
(∃ NroProv, Ciudad) (proveedor(NroProv, NombreProv, Ciudad) ∧ Ciudad ≠ 'Santa Cruz') }

b) { NombreCli | (∃ A, B) (cliente(A, NombreCli, B) ∧ (∃ C, D) (pedido(A, 50, C, D)))}

c) { Nombre1, Nombre2 | (∃ Cod1, Ciudad) (cliente(Cod1, Nombre1, Ciudad)
∧ (∃ Cod2) (cliente(Cod2, Nombre2, Ciudad) ∧ Cod1 ≠ Cod2)) }

d) { Nombre | (∃ Cod1) (cliente(Cod1, Nombre, 'Santa Cruz')) ∨
(∃ Cod2) (proveedor(Cod2, Nombre, 'Santa Cruz')) }

Silvia Alicia Gómez & Leticia Irene Gómez